Lista 3 – Algoritmos e Estrutura de Dados

Exercício 1. Considere a sequência de inserção em uma árvore binária de pesquisa sem balanceamento: 50, 30, 70, 20, 40, 60, 80.

a) Desenhe a árvore após cada inserção.

b) Para a árvore resultante, indique a raiz, as folhas e os nós internos.

c) Apresente os percursos em pré-ordem, em ordem (central) e pós-ordem.

d) Explique por que o percurso em ordem produz os valores em ordem crescente.

Exercício 2. Considere a seguinte estrutura:

```
typedef struct No {
    int valor;
    struct No *esq;
    struct No *dir;
} No;
```

Implemente as funções:

```
No *menorNo(No *raiz); // retorna o nó com o menor valor
No *maiorNo(No *raiz); // retorna o nó com o maior valor
int altura(No *raiz);  // retorna a altura do nó raiz
```

Observação: Adote a convenção de que uma árvore vazia tem altura `-1` e uma folha tem altura `0`.

Exercício 3. Ainda considerando a estrutura do exercício anterior, crie uma função que determina se uma determinada árvore está degenerada, ou seja, se qualquer nó da árvore possui no máximo 1 filho.

Exercício 4. Considere uma árvore AVL inicialmente vazia. Insira os valores abaixo exatamente na ordem apresentada, realizando o balanceamento sempre que necessário.

45, 20, 60, 10, 30, 50, 70, 5, 15, 25, 35, 55.

Insira os valores um a um, desenhando a árvore após cada inserção.